

## Issues in Artificial Neural Networks

### Overfitting in Neural Networks

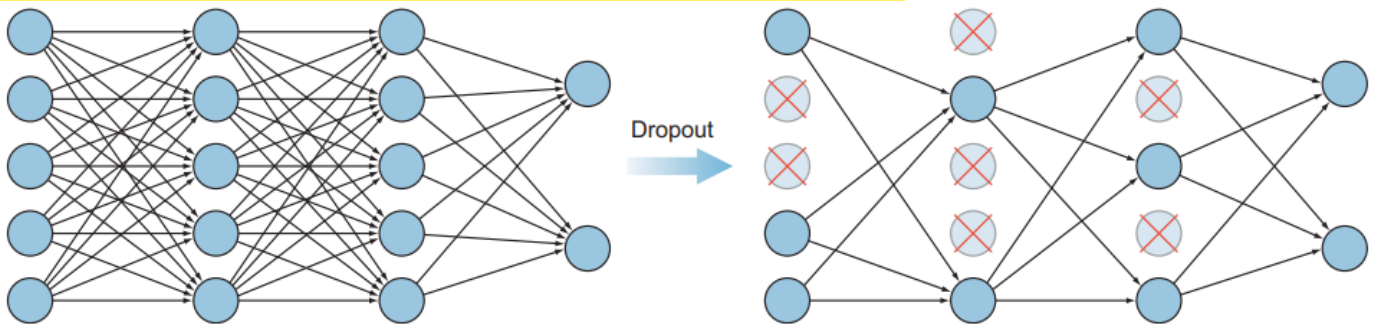
- Overfitting occurs when a model learns too well; such a model is not able to generalize well.
- Neural networks are particularly prone to overfitting.
- The more complex the model is, the more it is prone to overfitting.
- Some techniques to combat overfitting in neural networks are:
  - Early stopping
  - Data augmentation
  - L2 and L1 regularization
  - Dropout

Deep networks often have millions (or billions) of parameters. If your dataset isn't large or diverse enough, the model can essentially memorize the training data instead of learning general rules.

Early stopping, data augmentation, and regularization have already been discussed previously.

### Dropout

- A dropout layer is one of the most commonly used layers to prevent overfitting.
- It was proposed by Geoffrey Hinton in 2012 in his paper titled "Improving neural networks by preventing co-adaptation of feature detectors".
- Dropout randomly turns off a percentage of neurons (nodes) that make up a layer of your network.
- "Turns off" means these neurons are not included in a particular forward or backward pass.
- When a neuron is dropped out, the model shrinks and becomes less complex.



- Dropout technique makes a network resilient; it makes all the nodes work well as a team by making sure no node is too weak or too strong. Without dropout, neurons can become overly dependent on specific other neurons ("I only work if that feature detector fires"). With dropout, any neuron might disappear on a given step, so others must learn to perform well independently.
- The probability of choosing how many nodes should be dropped is a hyperparameter of the dropout function, called the dropout ratio  $p$  and its value ranges from 0 to 1. It is typically set to 0.5 (50%).
- After training, neurons don't get dropped anymore.
- For test data, every weight value is multiplied by keep probability  $(1 - p)$  after training.  $w_{test} = (1 - p)w_{trained}$
- Alternatively, we can divide each neuron's output by the keep probability during training (these alternatives are not perfectly equivalent, but they work equally well). With dropout, each neuron is kept with probability  $(1 - p)$  and dropped with probability  $p$ .
- Dropout techniques makes the neural network more robust that generalizes better.
  - Neurons trained with dropout cannot co-adapt with their neighboring neurons; they have to be as useful as possible on their own.
  - They end up being less sensitive to slight changes in the inputs.
- It generates more randomness.
  - A unique neural network is generated at each training step.
  - These neural networks are obviously not independent since they share many of their weights, but they are nevertheless all different.
  - So each iteration has a different set of nodes and this results in a different set of outputs.

At test time, we don't drop neurons anymore—we use the full network.  
But now there's a mismatch:  
During training, each neuron was active only  $\sim(1 - p)$  of the time  
During testing, it's active 100% of the time  
So if we do nothing, the total signal flowing through the network becomes larger than what the model saw during training.

- As a rule of thumb, the larger the network, the higher value of the dropout ratio.
- Dropout can be applied to both the hidden layers as well as the input layers.
- If you observe that the model is overfitting, you can increase the dropout rate. Conversely, you should try decreasing the dropout rate if the model underfits the training set.
- It can also help to increase the dropout rate for large layers, and reduce it for small ones.
- Moreover, many state-of-the-art architectures only use dropout after the last hidden layer, so you may want to try this if full dropout is too strong.
- Dropout does tend to significantly slow down convergence, but it usually results in a much better model when tuned properly. So, it is generally well worth the extra time and effort.

Why dropout slows convergence  
Dropout injects randomness into training:  
On every step, you're training a different sub-network (because some neurons are missing)  
The gradients become noisier  
The model can't rely on stable feature combinations  
So optimization takes longer to settle. In effect, the loss surface keeps "shifting" slightly each step, which slows down how quickly gradient descent converges.

## The Vanishing/Exploding Gradient Problem

- The backpropagation algorithm works by going from the output layer to the input layer, propagating the error gradient on the way.
- Once the algorithm has computed the gradient of the cost function with regards to each parameter in the network, it uses these gradients to update each parameter with a Gradient Descent step.
- Recall the error calculation and weight update for hidden layer  $L$ :  
$$\delta^{[L]} = w^{[L+1]} \delta^{[L+1]} * a'^{[L]}$$
$$w^{[L]} = w^{[L]} - \eta \frac{\partial J(w)}{\partial w^{[L]}} = w^{[L]} - \eta a^{[L-1]} \delta^{[L]}$$
- If the derivatives, calculated using the error values  $\delta^{[L]}$  are small then the gradients will decrease exponentially as propagated through the model until they eventually vanishes - the vanishing gradient problem.
- Alternatively, if the resulting gradients are large then the gradient will increase exponentially as propagated down the model until they eventually explode - the problem of exploding gradient.
- Reducing the number of layers can address both the issues, however, this solution will compromise model complexity, which could be harmful for complex mappings.

## Causes

- Historically, the issue of vanishing/exploding gradients was one of the major reasons behind training failure of deep networks.
- In 2010, a paper titled "Understanding the Difficulty of Training Deep Feedforward Neural Networks" by Xavier Glorot and Yoshua Bengio found two major suspects, which when used in combination caused this problem:
  - Weight initialization technique
  - Saturating activation functions (logistic sigmoid and tanh)
- In short, they showed that with this activation function and this initialization scheme, the variance of the outputs of each layer is much greater than the variance of its inputs.
- They identified that the signal needs to flow properly in both directions: in the forward direction when making predictions, and in the reverse direction when backpropagating gradients.
- We don't want the signal to die out, nor do we want it to explode and saturate.
- For the signal to flow properly, the authors argue that the variance of the outputs of each layer needs to be equal to the variance of its inputs, and also the gradients needs to have equal variance before and after flowing through a layer in the reverse direction.
- It is only possible if fan-in of the network is equal to its fan-out for each layer, but they proposed a good compromise in the form of some weight initialization strategies.
- Fan-in and Fan-out are defined for each layer as follows:
  - Fan-in ( $f_{in}$ ): The number of inputs to a perceptron (node) in the layer (number of nodes in the previous layer).
  - Fan-out ( $f_{out}$ ): The number of outputs from the layer (number of nodes in the current layer).
- Techniques to combat these gradient problems:
  - Careful initialization of weights (addresses both exploding and vanishing gradient issue)
  - Use of non-saturating activation function in hidden layers (addresses vanishing gradient issue)
  - Batch normalization or gradient scaling (addresses both exploding and vanishing gradient issue)
  - Gradient clipping (addresses exploding gradient issue)
- These methods only address the numerical stability of training deep neural network models and does not offer any general improvement in performance.

## Weight Initialization Techniques

- Weight initialization is a procedure to set the weights of a neural network to small random values that define the starting point for the optimization (learning or training) of the neural network model.
- Each time a neural network is initialized with a different set of weights, it starts the optimization process with a different starting point, and potentially results in a different final set of weights with different performance characteristics.
- Appropriate weight initialization is crucial in preventing exploding or vanishing gradients issues.
- If the weights are initialized correctly, then optimization of loss function will be fast.
- If all weights are initialized to 0 or same value then the gradient values for every weight will stay same in all subsequent iterations. This makes hidden layers symmetric and this process continues for all the iterations, making it no better than a linear model. [1]
- Typically, the biases for each unit are set to heuristically chosen constants (typically zero), and only the weights are randomly initialized.
- All the weights in a model are initialized to values drawn randomly from a Gaussian (normal) or uniform distribution.
- Traditionally, random initialization using a normal distribution with a mean of 0 and a standard deviation of 1 is used.
- The modern weight initialization techniques are based on the type of activation function used in the nodes that are being initialized, such as Sigmoid, Softmax Hyperbolic Tangent (Tanh) or ReLU.
- While these heuristics do not completely solve the exploding or vanishing gradients problems, they help to reduce it to a great extent.

### Weight Initialization for Sigmoid, Tanh and Softmax

- The current standard approach for initialization of the weights of neural network layers and nodes that use the Sigmoid or Tanh activation function is called **Glorot or Xavier initialization**.
- Named after Xavier Glorot, currently a research scientist at Google DeepMind.
- There are two versions of this weight initialization method, Xavier and Normalized Xavier.
- The **Xavier initialization method** is calculated as a random number with a uniform probability distribution ( $U$ ) between the range  $-(1/\sqrt{f_{in}})$  and  $(1/\sqrt{f_{in}})$ .

[1]  
What happens if all weights start the same? Suppose every neuron in a layer has identical weights, identical bias, Then for any input, all neurons produce the same output they receive the same gradient during backpropagation  
So after one update:  
all weights remain identical  
and this repeats forever

$$weight = U \left[ -\frac{1}{\sqrt{f_{in}}}, \frac{1}{\sqrt{f_{in}}} \right]$$

- The **Normalized Xavier initialization method** is calculated as a random number with a uniform probability distribution ( $U$ ) between the range  $-(\sqrt{6}/\sqrt{f_{in} + f_{out}})$  and  $(\sqrt{6}/\sqrt{f_{in} + f_{out}})$ .

$$weight = U \left[ -\frac{\sqrt{6}}{\sqrt{f_{in} + f_{out}}}, \frac{\sqrt{6}}{\sqrt{f_{in} + f_{out}}} \right]$$

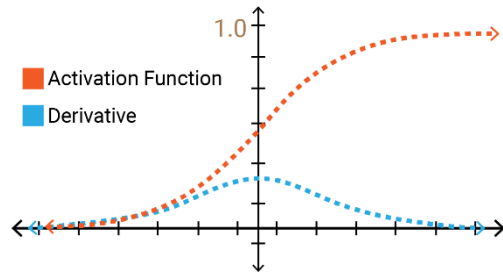
### Weight Initialization for ReLU and its Variants

- The current standard approach for initialization of the weights of neural network layers and nodes that use the rectified linear (ReLU) activation function is called **He initialization**.
- Named after Kaiming He, currently a research scientist at Facebook, and was described in the 2015 paper by Kaiming He, et al. titled Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.
- The **He initialization method** is calculated as a random number with a **Gaussian probability distribution ( $G$ )** with a mean of 0.0 and a standard deviation of  $\sqrt{2/f_{in}}$ .

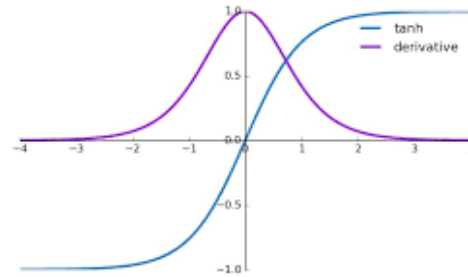
$$weight = G \left[ 0.0, \sqrt{\frac{2}{f_{in}}} \right]$$

- By default, Keras uses Glorot initialization with a uniform distribution.

## Saturating Activation Functions



**Sigmoid Function**



**Hyperbolic Tangent Function**

- Two commonly known saturating activation functions are Sigmoid and Hyperbolic Tangent (tanh).
- Derivative values of Sigmoid function ranges from 0 to 0.25, and that of Hyperbolic Tangent from 0 to 1.
- As can be seen the figures, when inputs become large (negative or positive), the function saturates at 0 or 1, with a derivative extremely close to 0.
- Thus when backpropagation kicks in, it has virtually no gradient to propagate back through the network, and what little gradient exists keeps getting diluted as backpropagation progresses down through the top layers, so there is really nothing left for the lower layers.
- A small gradient means that the weights and biases of the initial layers will not be updated effectively with each training session. Since these initial layers are often crucial to recognizing the core elements of the input data, it can lead to overall inaccuracy of the whole network.
- For shallow network with only a few layers that use these activations, this is not a big problem. However, when more layers are used, it can cause the gradient to be too small for training to work effectively.

## Non Saturating Activation Function

### The ReLU Activation Function

- Tanh and sigmoid functions are prone to the vanishing gradient problem, where gradients shrink drastically in backpropagation such that the network is no longer able to learn. ReLU avoids this by preserving the gradient since:
  - It does not saturate for positive values. Its linear portion (in positive input range) allows gradients to flow well on active paths of neurons and remain proportional to node activation.
  - It is an unbounded function i.e., no max value, a non-saturating function.
  - Additionally, it is quite fast to compute.
- Before ReLU, it was difficult to make deep neural networks; ReLU is now the new default for hidden layers.
- Unfortunately, the ReLU activation function is not perfect. In some scenarios it suffers from a problem known as the dying ReLUs: during training, some neurons effectively die, meaning they stop outputting anything other than 0.

## Batch Normalization

- Batch normalization (BN) or gradient scaling also help overcome the vanishing/exploding gradient problem.
- The problem arises when a large input space is mapped to a small one, causing the derivatives to vanish.
- BN (actually standardization around 0 mean) reduces this problem by simply normalizing the input so it does not reach the outer edges of the sigmoid function. All the layers can be batch normalized.
- Gradient scaling is rescaling of the error derivative, and consequently the weights, before propagating it backward through the network.

## Gradient Clipping

- Gradient Clipping is a popular technique to lessen the exploding gradients problem.
- It simply clips the gradients during backpropagation so that they never exceed some threshold.
- This technique is most often used in recurrent neural networks, as BN is tricky to use in RNNs. For other types of networks, BN is usually sufficient.